map from elements to indices.

But let’s see each operation in details, in table 2.4.

We have closely examined all operations. Yet, it’s worth spending a few more words on a couple of things.

For `insert` and `top`, the running time guarantee is amortized, not worst case. If a dynamic array is used to provide flexible sizes, some calls will require linear time to resize the array. It can be proven that, to fill a dynamic array with n elements, at most 2n swaps are needed. However, the worst-case logarithmic guarantee can be offered only if the heap size is set from the beginning. For this reason, and for allocation/garbage collection efficiency, in some languages (for instance Java), it is advisable to initialize your heaps to their expected size, if you have a reasonable estimate that will remain true for most of the container’s life.

The performance for `remove` and `updatePriority` relies on the efficient implementation of `contains`, in order to provide a logarithmic guarantee. To have efficient search, however, we need to keep a second data structure, besides the array, for fast indirection. The choice is between a hash table, or a bloom filter (see chapter 4).

In case either is used, the running time for contains is assumed to be constant, with a caveat: the hash for each element needs to be computable in constant time. Otherwise, we will need to take that cost as well into account in our analysis.